

Assignment 1: Building AI Apps with Gradio

This assignment has five tasks. Tasks 1 to 3 practise building interactive apps with Gradio. Tasks 4 and 5 use those skills to explore learned representations: word vectors, autoencoders and variational autoencoders.

The assignment is marked up to **100%**. Each task ends with a grading scheme, and the full marking rubrics are at the end of this document.

Task	Topic	Marks
1	GCD and LCM calculator	10
2	Wordle	20
3	Image gallery	20
4	Word analogy solver	20
5	Autoencoder and VAE lab	30
	Total	100

How to read this spec

Every task sorts its instructions into up to three lists:

- **Must.** Required. Anything missing or wrong here loses marks, as listed in the task's grading scheme.
- **Optional, but encouraged.** Worth trying if you have time. Skipping it costs nothing, and doing it earns no extra marks.
- **Suggestions.** One way to meet a Must, often the easiest. Any other way that meets the Must scores the same.

There is one special case: the **VAE in Task 5** is not a must, but 6 of Task 5's marks need it. Without the VAE, Task 5 is marked up to 24%.

The **mockups** are sketches of one possible layout, so everything in them counts as a suggestion unless a Must says otherwise. They are rough on purpose and are not screenshots of a real solution. Values shown as `?` or `0.____` are placeholders, not expected answers.

Some tasks need Gradio components we did not cover in class. Each one is linked to its documentation where it first appears.

General guidelines for designing Gradio apps

These are good practices for every app in this assignment. They count towards each task's UI/UX marks.

- **Label everything.** Every input and output component has a label that says what it is.
 - **Explain the app.** Each app has a short description telling the user what to do.
 - **Handle bad input.** Empty, invalid or unexpected input produces a helpful message, never a crash or a Python traceback. `gr.Error`, which we used in class, is the easiest way to do this.
 - **Show results the way a user would read them.** Each task says what this means for its own outputs, with an example of what loses marks.
-

Task 1: GCD and LCM Calculator (10 marks)

Objective: Build a Gradio app that takes two integers and shows both their greatest common divisor (GCD) and least common multiple (LCM) at the same time.

Must

1. Write a function `gcdlcm(x, y)` that returns two values: the GCD and the LCM.
2. Inputs may arrive as numbers or as text (if you use text boxes). Convert them to integers, and reject anything that is not an integer with a clear message.
3. The GCD is never negative. Handle zero correctly: the LCM is 0 when exactly one input is 0. When both inputs are 0, show a message instead of a number.
4. The interface has two inputs and **two separate, labelled outputs**, one for the GCD and one for the LCM. Returning both values as one Python tuple in a single box, such as `(6, 36)`, loses marks.
5. Test at least these pairs: (12, 18), (0, 5), (0, 0), (-8, 12), and one invalid input such as `abc`.

Suggestions

- `gr.Number` or `gr.Textbox` both work for the inputs. With text boxes you need to handle the conversion yourself.

Screenshots

Take **2**: one ordinary pair, such as (12, 18), and one corner case or invalid input, such as (0, 0) or `abc`.

Grading scheme

- **(4%) GCD and LCM implementation.** `gcdlcm` returns the correct GCD and LCM for ordinary positive integers.
 - **(3%) Gradio implementation and UI/UX.** Two inputs, two separate labelled outputs, a title and a short description.
 - **(3%) Error and corner-case handling.** Zeros, negatives, both zero and non-integer input all give a correct result or a clear message, never a crash.
-

Task 2: Wordle (20 marks)

Objective: Build a playable Wordle game. The computer picks a secret 5-letter word and the player has **6 attempts** to guess it. After each guess, every letter gets a colour:

- **Green:** the letter is in the word and in the right spot.
- **Yellow:** the letter is in the word but in a different spot.
- **Grey:** the letter is not in the word.

Repeated letters follow the standard Wordle rule. A letter is coloured green or yellow only as many times as it appears in the secret word, and green positions are decided before yellow ones. Work out carefully what this means for guesses and secrets that contain the same letter twice. Your function will be tested on such cases.

Task 2: Wordle (mockup) sketch only: layout and intent, not real output

Your guess

Submit guess

New game

Messages (a suggestion)

(win, lose, or why a guess was rejected goes here)

Example in this sketch:

the secret word is **CRANE**.

The player never sees it; it is shown here only to explain the colours.

H	O	U	S	E
R	E	A	C	T
T	R	A	C	E

Attempts left: 3 of 6

right spot

in word, wrong spot

not in word

Task 2 mockup

Mockup notes. On the left, the player types a guess. On the right, the board shows every guess so far as coloured tiles, with the attempts left underneath.

In this example the secret word is **CRANE**. The player never sees it; it is given here only so you can check the colours. After three guesses:

- **HOUSE:** only E is in CRANE, and it is in the same (last) spot, so E is green and the rest are grey.
- **REACT:** A is in the right spot (green). R, E and C are all in CRANE but in different spots (yellow). T is not in CRANE (grey).
- **TRACE:** R, A and E are in the right spots (green). C is in CRANE but not in the fourth spot (yellow). T is grey.

That leaves 3 of the 6 attempts.

Word list

Use exactly this list, for both the secret word and the valid guesses. Do not add or remove words.

```
WORDS = ""
about above actor adult after again agree ahead alarm album
alive allow alone apple apron arena argue arise arrow aside
audio award badge baker basic beach beard begin below bench
berry birth black blade blame blank blast bleed blend block
blood bloom board boost booth bound brain brand brave bread
break brick bride brief bring broad brown brush build bunch
buyer cabin cable candy cargo carry catch cause chain chair
chalk charm chart chase cheap check cheek cheer chess chest
chief child choir civic claim class clean clear clerk click
cliff climb clock close cloud coach coast color couch count
court cover crane crash cream crime crowd crown cycle daily
dance delay depth dirty doubt dozen draft drama dream dress
drink drive eager eagle early earth eerie eight elbow empty
enemy enjoy enter entry equal error event every exact exist
extra faith false fancy feast fence ferry fever field fifty
fight final first flame flash fleet flood floor flour fluid
focus force forty found frame fresh fruit funny geese ghost
giant glass globe glove grace grade grain grape grass great
green greet guard guess guest guide habit happy heart heavy
hello honey horse hotel house human humor ideal image index
inner input issue jelly jewel joint judge juice knife label
labor large laser later laugh layer learn lemon level light
limit llama lobby local logic loose lucky lunch magic major
maker march match mayor medal metal minor mixer model money
month moral motor mount mouse mouth movie music nerve never
night noble noise north novel nurse ocean offer often olive
onion opera orbit order other outer owner paint panel paper
party pasta peace pearl penny phone photo piano piece pilot
pizza place plain plane plant plate point pound power press
price pride prize proof proud queen quick quiet radio raise
rapid ratio reach react ready relax reply rider right river
robot rough round route royal rural salad sauce scale scene
scope score sense serve seven shade shake shape share sharp
sheep sheet shelf shell shift shine shirt shock shoot short
shout sight skill sleep slice small smart smile smoke snake
solid solve sound south space spare speak speed spend spice
spoon sport staff stage stair stamp stand start steam steel
stick still stone store storm story stove straw study style
sugar sunny super sweet table taste teach teeth thank theme
thick thing think three throw tiger title toast today tooth
topic total touch tower trace track trade train treat trend
trial truck trust truth twice uncle under unity upper upset
urban usual valid value video visit vital voice waste watch
water whale wheat wheel white whole woman world worry write
wrong young youth zebra
"".split()
```

Must

1. **Scoring function.** Write `score_guess(secret, guess)`, which returns a 5-character string made of G (green), Y (yellow) and X (grey). For example, `score_guess("crane", "trace")` returns "XGGYG". It must not depend on Gradio, because we will call it directly with our own test cases.
2. **Game rules.**

- A new game picks a random secret word from `WORDS`.
 - A guess that is not 5 letters, or not in `WORDS`, is rejected, the user is told why, and it **does not use up an attempt**.
 - Guesses are case-insensitive.
 - The game ends when the player guesses the word or uses all 6 attempts. When the player loses, show the secret word. Further guesses after the game ends are refused.
 - A **New game** button starts over.
3. **Board.** Show **all guesses so far** with each letter's colour, plus the number of attempts left. Full marks for the board need **real colours**, such as coloured tiles. A board that only marks the colours with letters or symbols earns at most half of the board marks, for example:

```
TRACE  XGGYG
T- R+ A+ C? E+
```

Why only half: such a board contains the right information, which earns the first half. But the player has to decode every letter against a legend (is + green or yellow?) and cannot read the whole board at a glance, which is the point of Wordle's colours and of this task's interface. The second half is for presenting the information the way a player reads it.

Suggestions

- Gradio has no ready-made tile component. Build the board as a small piece of HTML and show it with `gr.HTML`.
- The message box in the mockup is only one way to tell the user about rejected guesses and the end of the game. Pop-ups from `gr.Error`, `gr.Warning` or `gr.Info` work just as well.
- Your app has to remember the secret word and the guesses between clicks. A variable outside your functions works. `gr.State` is the tidier way.

Optional, but encouraged

- Use `gr.State` so that each browser gets its own game, instead of every player sharing one.

Screenshots

Take **2**: a game in progress with at least two guesses on the board, showing both green and yellow letters; and a finished game, won or lost, with its end message.

Grading scheme

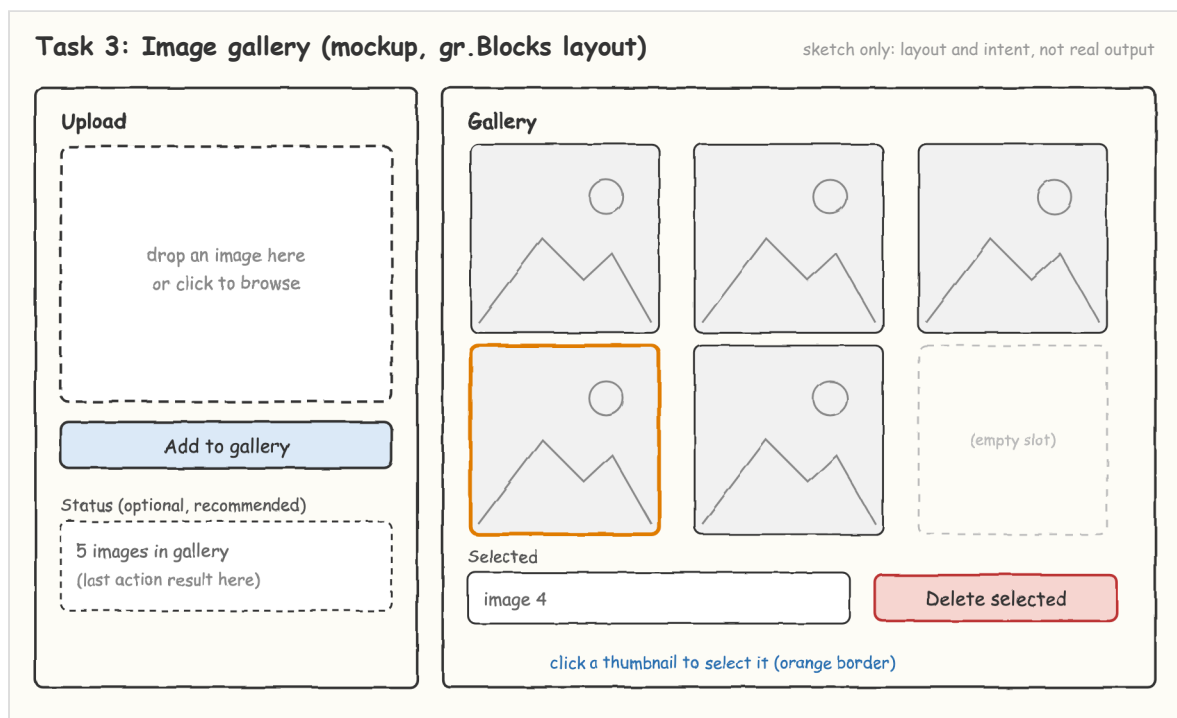
- **(4%) `score_guess`: basic scoring.** Correct G, Y and X for guesses and secrets without repeated letters.
- **(3%) `score_guess`: repeated letters.** Correct on our test cases where the guess or the secret repeats a letter.
- **(4%) **Game flow.**** Random secret from `WORDS`, 6 attempts counted correctly, win and loss detected, secret shown on a loss, New game resets everything.
- **(3%) **Input validation.**** Wrong length and words not in the list are rejected without using an attempt; case-insensitive; guesses after the game ends are refused.

- **(4%) Board display.** All guesses so far shown as coloured tiles (for example with `gr.HTML`), with attempts left.
- **(2%) UI/UX.** Clear labels, a short how-to-play description, and the user can tell when a guess is rejected or the game ends.

Task 3: Image Gallery (20 marks)

Objective: Build a small image gallery app where the user can **list**, **upload** and **delete** images.

In class we saw that `gr.Gallery` displays a list of image file paths. How your app keeps that list and updates it after each operation is for you to design.



Task 3 mockup

Mockup notes. On the left, the user uploads an image and adds it. On the right, the gallery shows every image; the orange border marks the one the user clicked, which the Delete button removes.

Must

1. **List.** The gallery always shows every image currently stored. An empty gallery shows a sensible message, not an error. Showing the stored file paths as text, such as `['/tmp/gradio/3f2a.../cat.png', ...]`, instead of (or as well as) the images loses marks.
2. **Upload.** The user picks any image, and it is added to the gallery as a new item.
3. **Select and delete.** The user selects one image in the gallery by clicking it, and a delete action removes that image and only that image.
4. **Edge cases.** Pressing upload with no file chosen, or delete with nothing selected, gives a message and changes nothing. After a delete, the app must not keep "remembering" a selection that now points at a different image.
5. **Order.** Decide where new images appear (first or last) and say so in your description. It must stay consistent.

6. **Layout.** Full marks for the interface need **separate controls for each operation**, for example `gr.Blocks` with separate buttons. A single `gr.Interface` with a `gr.Dropdown` to choose the operation is accepted but earns partial marks.

Suggestions

- Lay it out with `gr.Blocks`, a `gr.Row` and two `gr.Columns`, as in class and in the mockup.
- To find out which image was clicked, listen to the gallery's `select` event and read the index from `gr.SelectData`. See also the [gr.Gallery documentation](#).

Optional, but encouraged

- A **status box** that tells the user something useful after each action, such as how many images there are now or which image was just deleted.
- Give each browser session its own gallery with `gr.State`, or save uploads to a folder so the gallery survives a restart.

Screenshots

Take **2**: the gallery with at least three uploaded images and one of them selected; and the same gallery after that image was deleted.

Grading scheme

- **(4%) Upload.** A chosen image is added to the gallery and appears immediately.
- **(3%) List and order.** The gallery always shows every stored image, the empty state is sensible, and the order is consistent and stated in the description.
- **(3%) Select.** Clicking a thumbnail tells the app which image the user means.
- **(4%) Delete.** The delete action removes exactly the selected image and the gallery updates.
- **(3%) Error and corner-case handling.** Upload with no file and delete with nothing selected give a message and change nothing; no stale selection after a delete.
- **(3%) UI/UX.** Separate labelled controls for each operation, a short description, and the user can tell what each action did.

Task 4: Word Analogy Solver (20 marks)

Objective: Use pre-trained word vectors to solve analogies of the form

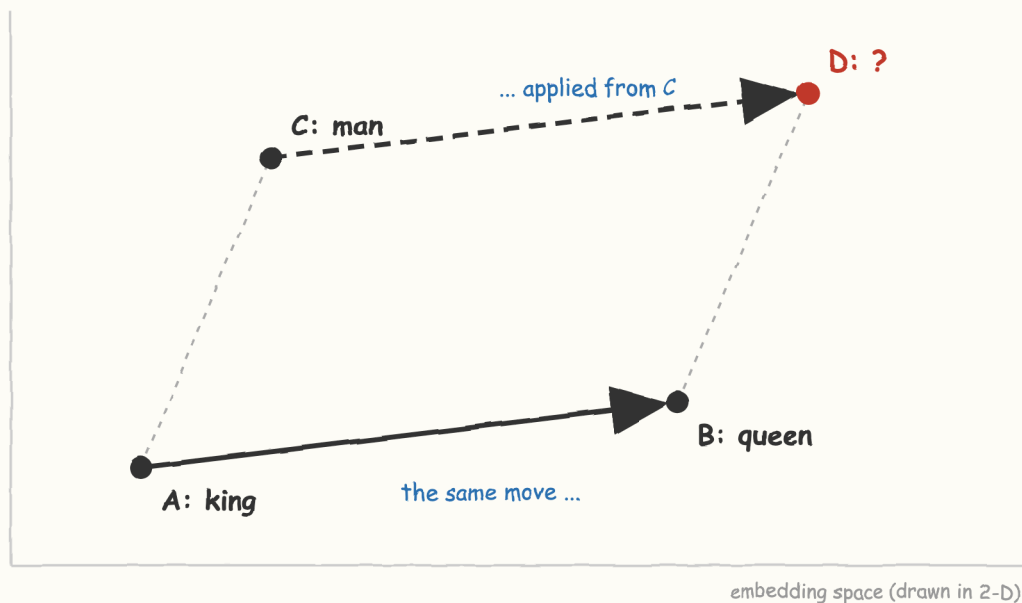
A is to B as C is to D

The user types three words, A, B and C, and your app predicts the missing word D. For example, *king* is to *queen* as *man* is to ?.

Word vectors place words as points in a space where directions can carry meaning. The figure below shows the idea in two dimensions. Work out from the figure how to compute the point where D should be, using the vectors of A, B and C, and then find the words nearest to that point.

Task 4: the parallelogram idea

figure



Parallelogram figure

Figure notes. The solid arrow is the move from A to B. The dashed arrow is the same move, started from C. D is where it lands.

Task 4: Word analogy solver (mockup)

sketch only: layout and intent, not real output

A is to B as C is to D. Short instructions and an example go here.

A: source 1

type a word

B: what A becomes

type a word

C: source 2

type a word

Solve

Clear

(optional) warnings, or use `gr.Error`

Predicted D

???

Top 5 candidates (gr.Label)

word 1		0.00
word 2		0.00
word 3		0.00
word 4		0.00
word 5		0.00

Task 4 mockup

Mockup notes. On the left, the three input words. On the right, the predicted D and the top 5 candidates with their scores.

Must

1. **Model.** Load `glove-wiki-gigaword-100` with `gensim.downloader`. The file is about 128 MB. Load it once, in its own cell, before building the app: Colab then downloads it only once per session, and

every query reuses the loaded model. Loading it inside your Gradio function would reload it (and may download it again) on every click. Everyone uses the same model so results can be compared.

2. **Target vector.** Compute the target point yourself with vector arithmetic in your own code, following the figure. A library function that solves the whole analogy for you does not count for this part.
3. **Retrieval.** Find the 5 words closest to the target by cosine similarity, **excluding A, B and C**, ranked from most to least similar, each with its similarity score. The top word is your predicted D. The key function is the loaded model's `similar_by_vector(vector, topn=...)`, which returns the `topn` nearest words to any vector as (word, cosine similarity) pairs. It does not know which words you typed, so A, B or C can appear in its results.
4. **Input handling.** Lowercase the inputs. If a word is not in the model's vocabulary, say which one. If two of the inputs are the same word, warn the user.
5. **Display.** Three clearly labelled inputs. Show the predicted D prominently and the top 5 in a `gr.Label` component. Printing the raw result of your search in a text box, such as `[('paris', 0.71), ('rome', 0.68), ...]` or a JSON string, loses marks. The description explains the "A is to B as C is to D" pattern with an example.
6. **Testing.** Try at least five analogies of different kinds (for example gender, country and capital, verb tense, comparatives). Include one that **fails**, and add a sentence on why you think it fails.

Suggestions

- The warnings box in the mockup is only one way to report problems. For an unknown word, raising `gr.Error` with a message that names the word is the simplest option. For something allowed but suspicious, such as A and B being the same word, `gr.Warning` shows a pop-up without stopping the app.
- `gr.Label` accepts a dictionary of word to score and draws the bars for you.

Screenshots

Take **2**: one analogy with its predicted D and top 5; and the message for a word that is not in the vocabulary. Your five tested analogies can be recorded as screenshots or as a table.

Grading scheme

- **(2%) Loading the model.** `glove-wiki-gigaword-100` loads with `gensim.downloader` and is ready for queries.
 - **(5%) Target vector.** The target point is computed from A, B and C with vector arithmetic in your own code, following the figure.
 - **(4%) Retrieval and ranking.** The 5 nearest words by cosine similarity, ranked, each with its score; the top word is D.
 - **(2%) Excluding the inputs.** A, B and C never appear among the candidates.
 - **(3%) Input handling.** Inputs lowercased; unknown words named in a clear message (for example with `gr.Error`); repeated inputs warned about.
 - **(2%) UI/UX.** Three labelled inputs, D shown prominently, the top 5 in `gr.Label`, and a description of the pattern with an example.
 - **(2%) Tested analogies.** At least five analogies of different kinds, including one failure with a sentence on why it fails.
-

Task 5: Autoencoder and VAE Lab (30 marks)

Objective: Train an autoencoder (AE) on MNIST handwritten digits, then build a Gradio app with four separate interfaces that let a user explore what the model learned. Then, for full marks, add a variational autoencoder (VAE) and a fifth interface that walks through its latent space.

The VAE is not a must, but without it Task 5 is marked up to 24%. The remaining 6% is only available with a VAE (see “For full marks: the VAE” below). Everything under Must works with a plain AE.

Must: training (in your notebook, before the app)

- The AE encodes a 28×28 grayscale digit into a latent vector of **at most 16 numbers** and decode it back to an image. The architecture is your choice.
- MNIST digits are white on black. Whatever pixel range you train on (0 to 255 or 0 to 1), every input in your app must be converted to the same range, and outputs converted back for display.
- Train before launching the app. Do not train inside a Gradio function.

Must: the app

The app has **four separate interfaces**, described below: Reconstruct, Draw, Noise and Garbage in. Each has its own inputs, outputs and button. They can be four individual Gradio apps, or one app with a tab for each (strongly recommended, see below).

1. **Reconstruct.** The user uploads a digit image. Resize to 28×28 and normalise inside your function (the user can upload any size). Show the latent, the reconstruction and the reconstruction error (MSE). The latent may be shown as rounded numbers in a text box, such as 0.12, -1.30, 0.85, Printing the raw array or tensor, such as `array([[ 0.1234567 , -1.3012345 , ...]], dtype=float32)`, loses marks.
2. **Draw.** The user draws a digit in `gr.ImageEditor`. Take the composite image from the editor, convert it to grayscale, detect the background colour (drawings are usually dark on white) and invert if needed so it matches MNIST, then resize and reconstruct. Show the 28×28 image the model actually receives next to the reconstruction.
3. **Noise.** A slider sets a noise level σ . Add Gaussian noise (`np.random.normal(0,  $\sigma$ , shape)`) to the normalised input and clip the result to the valid range. Show the clean input, the noisy input and the reconstruction of the noisy input.
4. **Garbage in.** One image input, and its reconstruction and MSE as outputs. Run it once for each test input: **one real digit and at least three things that are not digits**. Take a screenshot of every run (see Screenshots). Then, under the app, answer in a few sentences:
 - What does the model produce when the input is not a digit, and why?
 - Could the MSE tell you whether an input is a digit? Use your numbers.

For full marks: the VAE (6 marks)

Not a must, but these 6 marks need it.

1. **Train a VAE** alongside your AE, with a latent of at most 16 numbers. Its encoder outputs a mean μ and a log-variance $\log \sigma^2$, a latent z is sampled from them with the reparameterisation trick, and the loss adds a KL divergence term to the reconstruction loss. If you adapt code from an example, cite it in a comment.

2. **Reconstruct with either model.** In Reconstruct, the user chooses AE or VAE. For the VAE, show μ and the sampled z **separately**, as rounded numbers like the latent.
3. **Interpolate (a fifth interface).** The user uploads two digits, X and Y. Encode both with the VAE, move in straight-line steps from X's latent to Y's latent (the endpoints are exactly X and Y), and decode each step into a path of images. Check that the first and last images match the reconstructions of X and Y. Say whether you interpolate μ or the sampled z , and why. In a few sentences, describe the in-between images: do they look like real digits, and why do you think so?

Suggestions

- Choose between AE and VAE with a [gr.Radio](#).
- The value of `gr.ImageEditor` is a dictionary, and the finished drawing is under one of its keys: check the documentation for which one.
- Update the Noise output whenever the slider moves, so the user does not need to press a button.
- For Garbage in, try a letter, a photo, a blank image, or a digit drawn black on white. Any non-digit inputs are fine.
- The [Keras VAE example](#) is a good reference for the VAE.

Optional, but encouraged

- **Strongly recommended:** put all the interfaces in one `gr.Blocks` app, with one [gr.Tab](#) each, as in the mockups below. The user then has one app to open and can switch between experiments without restarting anything.
- Show the latent, μ and z as a small chart instead of numbers.

Mockups

Task 5: Reconstruct tab (mockup) sketch only: layout and intent, not real output

Reconstruct
Draw
Noise
Garbage in
Interpolate (VAE)

Model

(·) AE
() VAE

Latent (at most 16 numbers)

0.__, 0.__, 0.__, 0.__, 0.__, 0.__, ...

Reconstruction

?

Upload a digit

drop image

Encode + decode

VAE only: μ

0.__, 0.__, 0.__, 0.__, 0.__, 0.__, ...

VAE only: sampled z

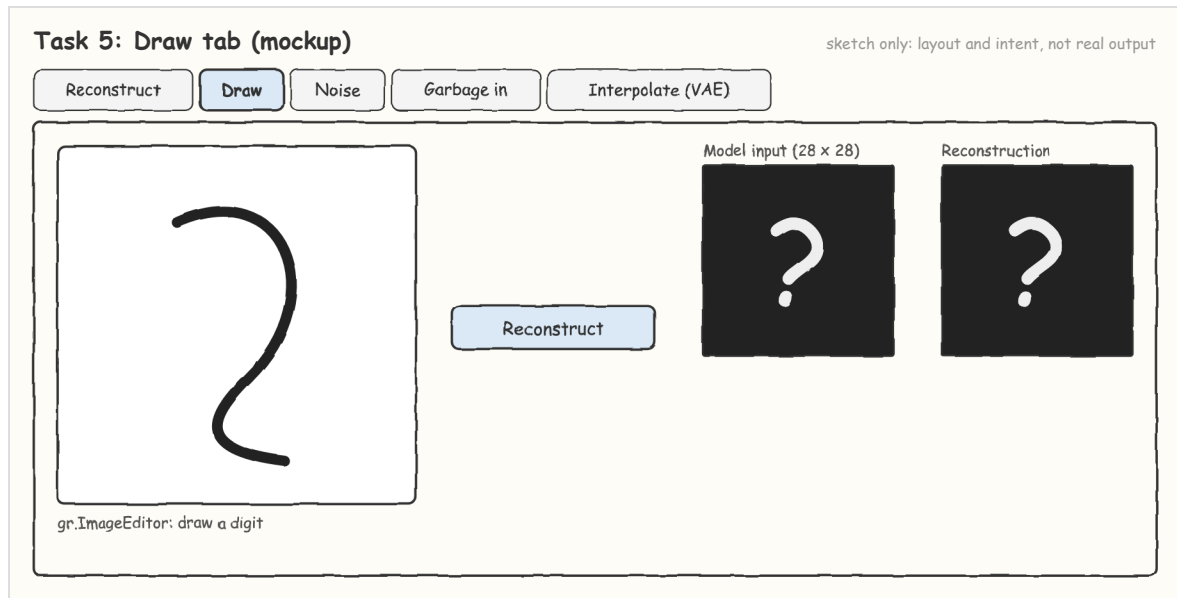
0.__, 0.__, 0.__, 0.__, 0.__, 0.__, ...

Reconstruction error (MSE)

0.0__

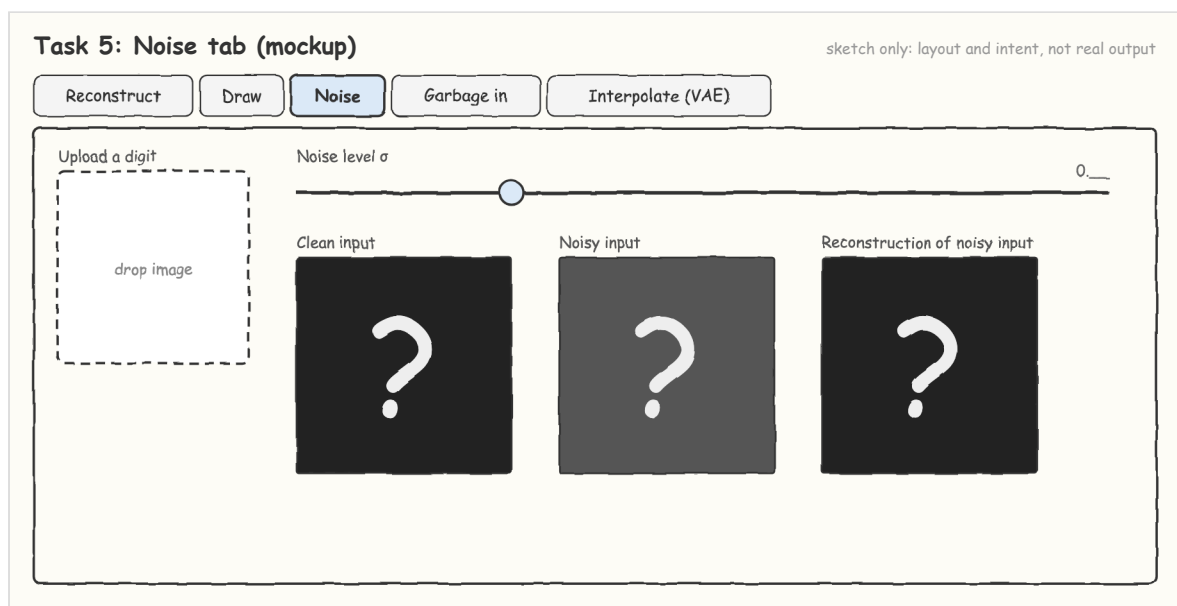
Task 5 mockup: Reconstruct

Reconstruct. The model choice at the top left, the latent, μ and z as text boxes in the middle, the reconstruction and its MSE on the right. The μ and z boxes only matter when the VAE is selected.



Task 5 mockup: Draw

Draw. The drawing area on the left. “Model input” is the 28×28 image after your preprocessing, shown next to the reconstruction.



Task 5 mockup: Noise

Noise. One uploaded digit, the σ slider, and the clean input, noisy input and reconstruction side by side.

Task 5: Garbage in tab (mockup) sketch only: layout and intent, not real output

Reconstruct Draw Noise **Garbage in** Interpolate (VAE)

Upload anything (a digit, a letter, a photo, ...)

drop image

Encode + decode

Reconstruction



Reconstruction error (MSE)

0.0

Run it once per test input:
one real digit and at least
three things that are not digits.

Task 5 mockup: Garbage in

Garbage in. One input, its reconstruction and its MSE. The same interface is run once per test input.

Task 5: Interpolate tab, VAE only (mockup) sketch only: layout and intent, not real output

Reconstruct Draw Noise Garbage in **Interpolate (VAE)**

Image X

Image Y

Steps 7

VAE path: X to Y



X y

Run

Task 5 mockup: Interpolate

Interpolate (VAE only). Two uploaded digits, a slider for the number of steps, and the decoded path from X to Y. The first image corresponds to X and the last to Y.

Screenshots

- **Reconstruct:** 2, with two different digits.
- **Draw:** 2, with two different drawings.
- **Noise:** 2, the same digit at a low and a high σ .
- **Garbage in:** one per test input, so at least 4.
- **VAE (if you built it):** 1 of Reconstruct with the VAE selected, showing μ and z , and 2 of Interpolate with two different pairs of digits.

Grading scheme

- **(3%) AE training.** An AE trained on MNIST with a latent of at most 16 numbers, trained before the app starts.

- **(2%) Reconstruct: preprocessing.** Uploads of any size are converted to grayscale, resized to 28×28 and normalised to the training range.
- **(4%) Reconstruct: outputs.** The latent (numbers in a text box are fine), the reconstruction and the MSE are shown and labelled.
- **(3%) Draw: preprocessing.** The composite is taken from `gr.ImageEditor`, converted to grayscale, the background is detected and inverted when needed, then resized.
- **(2%) Draw: display.** The 28×28 model input is shown next to the reconstruction.
- **(2%) Noise: noise.** Gaussian noise with the slider's σ is added in the normalised range and clipped.
- **(2%) Noise: display.** A σ slider, and the clean input, noisy input and reconstruction shown side by side.
- **(4%) Garbage in: experiments.** An interface showing the reconstruction and MSE for an uploaded image, run on one real digit and at least three non-digit inputs.
- **(2%) Garbage in: written answers.** Both questions answered using your own MSE numbers.

Subtotal without the VAE: 24%.

- **(2%) VAE: model.** The encoder outputs μ and $\log \sigma^2$, z is sampled with the reparameterisation trick, and the loss includes the KL term.
- **(1%) VAE: display.** A model choice in Reconstruct, with μ and the sampled z shown separately.
- **(3%) VAE: interpolation.** Straight-line steps between the two VAE latents, endpoints matching the reconstructions of X and Y , the choice of μ or z stated with a reason, and the in-between images described.

Screenshots

Every task lists the screenshots it needs in its own **Screenshots** subsection. For all of them:

- Take them of your running app, not of the mockups, with your own inputs.
- Put them in your notebook under the task they belong to, or in one PDF.
- Give each a one-line caption saying what it shows.

Deduction. Missing screenshots cost marks. For each task, or each Task 5 interface, whose required screenshots are missing or do not show what its Screenshots subsection asks for, the marks for its interface are halved: the UI/UX item in Tasks 1 to 4, and that interface's display and output items in Task 5. Screenshots that are not of your own running app count as missing.

Submission Guidelines

Deadline: 11:59 PM, Friday 9 October 2026.

Late submissions lose 20% per day, up to 5 days. After that, submissions are not accepted. After you submit, you will get feedback on Moodle. You will then have a chance to improve your work and submit a final version.

Submission requirements:

- Submit through Moodle, and click the "Submit" button. I grade only submitted work.

- Include the Colab link to your project, or attach the `.ipynb` file if you used Jupyter Notebook. Make sure a Colab link is shared so that I can open it.
- Put **all five tasks** in the same notebook.
- Attach screenshots (images or a PDF) as described above.
- Do not use the “Submission comments” box to contact me about your submission. I do not read it. I only read the submission text and the attached files.

Notes:

- Plagiarism results in a grade of zero.
- The assignment is individual.
- Coding agents and chatbots are allowed. State in comments which tools you used and for what. You are responsible for every line: you must be able to explain your code and your design choices.
- Cite any external source you used for ideas or code.

Rubrics

Each item in the grading schemes is marked at one of the levels below. The marks for each task add up to its total.

Task 1: GCD and LCM Calculator (10 marks)

Item	Marks	Levels
GCD and LCM implementation	4	4: <code>gcd lcm</code> returns the correct GCD and LCM for ordinary positive integers. 2: correct for some inputs only. 0: missing or not working.
Gradio implementation and UI/UX	3	3: two inputs, two separate labelled outputs, a title and a short description. 1: works, but outputs unlabelled or combined into one box. 0: missing or not working.
Error and corner-case handling	3	3: zeros, negatives, both zero and non-integer input all give a correct result or a clear message, never a crash. 1: some cases handled, others wrong or crashing. 0: missing or not working.

Task 2: Wordle (20 marks)

Item	Marks	Levels
<code>score_guess</code> : basic scoring	4	4: correct G, Y and X for guesses and secrets without repeated letters. 2: mostly correct, with some positions wrong. 0: missing or not working.
<code>score_guess</code> : repeated letters	3	3: correct on our test cases where the guess or the secret repeats a letter. 1: some repeated-letter cases correct. 0: missing or not working.
Game flow	4	4: random secret from <code>WORDS</code> , 6 attempts counted correctly, win and loss detected, secret shown on a loss, New game resets everything. 2: playable, but one or two of these missing or wrong. 0: missing or not working.
Input validation	3	3: wrong length and words not in the list are rejected without using an attempt; case-insensitive; guesses after the game ends are refused. 1: some checks missing. 0: missing or not working.

Item	Marks	Levels
Board display	4	4: all guesses so far shown as coloured tiles (for example with <code>gr.HTML</code>), with attempts left. 2: colours marked only with letters or symbols, or only the last guess shown. 0: missing or not working.
UI/UX	2	2: clear labels, a short how-to-play description, and the user can tell when a guess is rejected or the game ends. 1: works, but unclear or missing feedback. 0: missing or not working.

Task 3: Image Gallery (20 marks)

Item	Marks	Levels
Upload	4	4: a chosen image is added to the gallery and appears immediately. 2: added, but only appears after another action, or overwrites existing images. 0: missing or not working.
List and order	3	3: the gallery always shows every stored image, the empty state is sensible, and the order is consistent and stated in the description. 1: shown, but empty state or order is wrong. 0: missing or not working.
Select	3	3: clicking a thumbnail tells the app which image the user means. 1: the selection sometimes points at the wrong image. 0: missing or not working.
Delete	4	4: the delete action removes exactly the selected image and the gallery updates. 2: deletes, but sometimes the wrong image. 0: missing or not working.
Error and corner-case handling	3	3: upload with no file and delete with nothing selected give a message and change nothing; no stale selection after a delete. 1: some cases handled, others crash or act wrongly. 0: missing or not working.
UI/UX	3	3: separate labelled controls for each operation, a short description, and the user can tell what each action did. 1: works, but a drop-down of operations or no sign that an action happened. 0: missing or not working.

Task 4: Word Analogy Solver (20 marks)

Item	Marks	Levels
Loading the model	2	2: <code>glove-wiki-gigaword-100</code> loads with <code>gensim.downloader</code> and is ready for queries. 0: missing or wrong.
Target vector	5	5: the target point is computed from A, B and C with vector arithmetic in your own code, following the figure. 2: arithmetic attempted with the wrong combination. 0: missing or not working.
Retrieval and ranking	4	4: the 5 nearest words by cosine similarity, ranked, each with its score; the top word is D. 2: results shown, but unranked or without scores. 0: missing or not working.
Excluding the inputs	2	2: a, B and C never appear among the candidates. 0: missing or wrong.
Input handling	3	3: inputs lowercased; unknown words named in a clear message (for example with <code>gr.Error</code>); repeated inputs warned about. 1: some cases handled, others crash. 0: missing or not working.

Item	Marks	Levels
UI/UX	2	2: three labelled inputs, D shown prominently, the top 5 in <code>gr.Label</code> , and a description of the pattern with an example. 1: works, but raw output or no explanation. 0: missing or not working.
Tested analogies	2	2: at least five analogies of different kinds, including one failure with a sentence on why it fails. 1: fewer analogies, or no failure explained. 0: missing or not working.

Task 5: Autoencoder and VAE Lab (30 marks)

Item	Marks	Levels
AE training	3	3: an AE trained on MNIST with a latent of at most 16 numbers, trained before the app starts. 1: trained, but the latent is larger than 16. 0: missing or not working.
Reconstruct: preprocessing	2	2: uploads of any size are converted to grayscale, resized to 28×28 and normalised to the training range. 1: one step missing or wrong. 0: missing or not working.
Reconstruct: outputs	4	4: the latent (numbers in a text box are fine), the reconstruction and the MSE are shown and labelled. 2: one output missing, or the latent printed as a raw array. 0: missing or not working.
Draw: preprocessing	3	3: the composite is taken from <code>gr.ImageEditor</code> , converted to grayscale, the background is detected and inverted when needed, then resized. 1: works, but no inversion or no background check. 0: missing or not working.
Draw: display	2	2: the 28×28 model input is shown next to the reconstruction. 1: only the reconstruction shown. 0: missing or not working.
Noise: noise	2	2: Gaussian noise with the slider's σ is added in the normalised range and clipped. 1: noise added in the wrong range or not clipped. 0: missing or not working.
Noise: display	2	2: a σ slider, and the clean input, noisy input and reconstruction shown side by side. 1: an image missing. 0: missing or not working.
Garbage in: experiments	4	4: an interface showing the reconstruction and MSE for an uploaded image, run on one real digit and at least three non-digit inputs. 2: fewer than three non-digit inputs, or MSE missing. 0: missing or not working.
Garbage in: written answers	2	2: both questions answered using your own MSE numbers. 1: answered without using the evidence. 0: missing or not working.
VAE: model	2	2: the encoder outputs μ and $\log \sigma^2$, z is sampled with the reparameterisation trick, and the loss includes the KL term. 1: μ and $\log \sigma^2$ present, but no KL term or no sampling. 0: missing or not working.
VAE: display	1	1: a model choice in Reconstruct, with μ and the sampled z shown separately. 0: missing or wrong.
VAE: interpolation	3	3: straight-line steps between the two VAE latents, endpoints matching the reconstructions of X and Y, the choice of μ or z stated with a reason, and the in-between images described. 1: path shown, but endpoints wrong or unchecked, or no reason or description. 0: missing or not working.